IPC 多核开发手册

Revision History

Draft Date	Revision No.	Description
2024/10/22	V1.0	1. 初始版本。

目　　录

前　言 3
1 IPC 多核通信机制 5
1.1 IPC 简介 5
1.2 Cortex-M7 核心程序加载启动流程 6
1.3 Linux 应用层空间与 Cortex-M7 核心通信 7
2 openamp-demos 9
2.1 rpmsg_lite_led_flash 案例 11
2.1.1 案例说明 11
2.1.2 案例测试 13
2.1.3 案例编译 17
2.1.4 关键代码 18
2.2 rpmsg_lite_pingpong_rtos 案例 21
2.2.1 案例说明 21
2.2.2 案例测试 22
2.2.3 案例编译 25
2.2.4 关键代码 25
2.3 rpmsg_lite_str_echo_rtos 案例 30
2.3.1 案例说明 30
2.3.2 案例测试 31
2.3.3 案例编译 35
2.3.4 关键代码 36
更多帮助 39

前　言

本文档主要说明创龙科技 TLIMX8MP-EVM 评估板基于 IPC 的多核开发方法。适用开发环境如下：

Windows 开发环境：Windows 7 64bit、Windows 10 64bit

虚拟机：VMware15.5.5

Linux 开发环境：Ubuntu20.04.6 64bit

U-Boot：U-Boot-2022.04

Kernel：Linux-5.15.71-rt51

SDK 开发包：SDK_2_12_0_MIMX8ML8xxxKZ

交叉编译工具链：gcc-arm-none-eabi-10.3-2021.10-x86_64-linux

i.MX 8M Plus 为 SoC 多核处理器，一般情况下，Cortex-A53 核心运行 Linux 系统，Cortex-M7 核心运行裸机或 RTOS 程序。

下图为 i.MX 8M Plus 芯片的框图，i.MX 8M Plus 内部高度集成了包括 Cortex-A53、Cortex-M7 以及 DSP 在内的多种处理器单元。这些处理器单元通过协同工作，共同实现最终的应用设计目标。

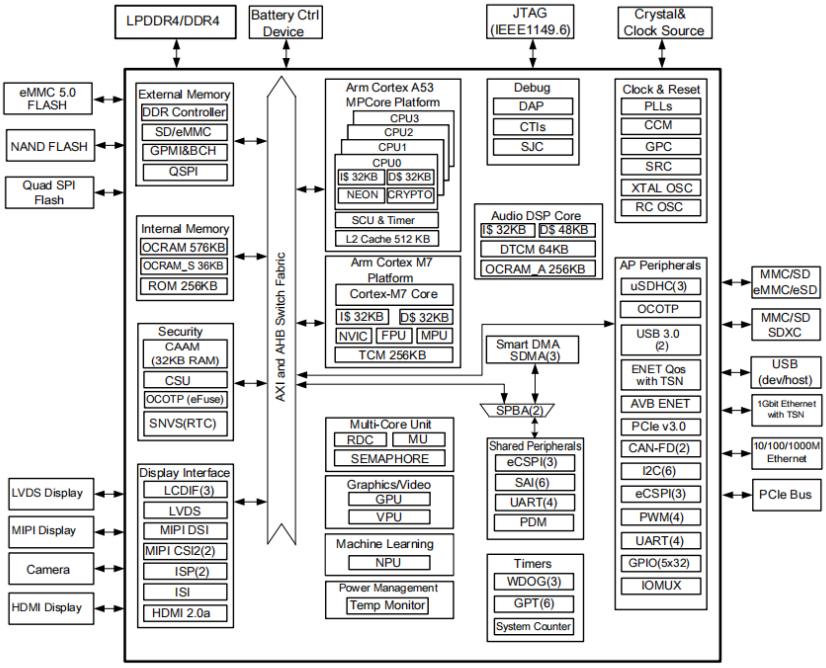

Figure 1-1. Block Diagram

图 1

IPC 多核开发的 openamp-demos 案例（Cortex-M7 与 Cortex-A53 核心进行核间通信的 FreeRTOS 案例）位于“4-软件资料\Demo\openamp-demos\”目录下。案例包含 rpmsg_lite_led_flash、rpmsg_lite_pingpong_rtos、rpmsg_lite_str_echo_rtos 案例。

表 1

案例				说明
openamp-demos	rpmsg_lite_led_flash	bin	rpmsg_lite_led_flash	Linux 应用程序镜像
			rpmsg_lite_led_flash_rtos.bin	运行在 TCM 的 Cortex-M7 核心镜像
			rpmsg_lite_led_flash_rtos_ddr.bin	运行在 DDR 的 Cortex-M7 核心镜像
		project		Cortex-M7 核心例程源码目录
		src	Makefile	Makefile 编译脚本
			rpmsg_lite_led_flash.c	Linux 应用程序源码
	rpmsg_lite_pingpong_rtos	bin	rpmsg_lite_pingpong_rtos.bin	运行在 TCM 的 Cortex-M7 核心镜像
			rpmsg_lite_pingpong_rtos_ddr.bin	运行在 DDR 的 Cortex-M7 核心镜像
		project		Cortex-M7 核心例程源码目录
	rpmsg_lite_str_echo_rtos	bin	rpmsg_lite_str_echo	Linux 应用程序镜像
			rpmsg_lite_str_echo_rtos.bin	运行在 TCM 的 Cortex-M7 核心镜像
			rpmsg_lite_str_echo_rtos_ddr.bin	运行在 DDR 的 Cortex-M7 核心镜像
		project		Cortex-M7 核心例程源码目录
		src	Makefile	Makefile 编译脚本
			rpmsg_lite_str_echo.c	Linux 应用程序源码

1 IPC 多核通信机制

1.1 IPC 简介

IPC 全称处理器间通信(Inter Processor Communications)，在系统级芯片(SoC)内部实现了多个处理器核心之间的有效沟通。其中，RPMsg(Remote Processor Messaging)作为 IPC 的一种机制，是异构多核处理系统(AMP)中用于核间通信的一种标准二进制接口。其使得主处理器与远程处理器（例如 DSP、MCU 等）能够通过共享内存实现高效的数据传输。RPMsg 基于 Virtio 框架构建，利用 Virtio 提供的通信机制，实现了低延迟、高性能的核间通信。

为了满足资源受限的嵌入式系统的需求，NXP Semiconductors 开发了 RPMsg-lite，这是 RPMsg 的轻量级实现。RPMsg-lite 在 BSD 兼容许可下发布，为开发者提供了既灵活又高效的核间通信解决方案。与 RPMsg 相比，RPMsg-lite 在代码规模上大幅缩减，简化了 API，并增强了模块化特性，使其更加适用于资源受限的环境。

用户可通过两种方式实现与 Cortex-M7 的 IPC 通信。

（1） 在应用层（用户空间）代码访问"/dev/ttyRPMsgxx"设备节点（x 表示设备号），调用系统 read、write 接口，将会调用驱动层的"imx_rpmsg_tty driver -> RPMsg driver -> RemoteProc driver -> VirtIO driver -> Mail Box Driver"传递数据，实现 IPC 通信功能。

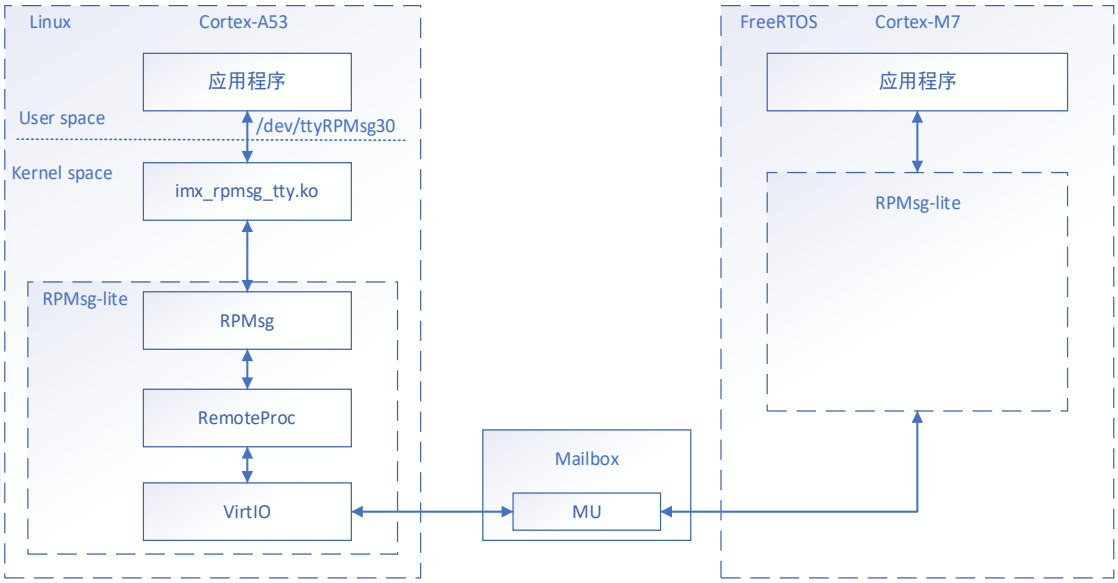

图 2

（2） 在驱动层（内核空间）使用 imx_rpmsg_pingpong 或 imx_rpmsg_tty 驱动直接或间接调用 RPMsg-lite 接口，调用"RPMsg driver -> RemoteProc driver -> VirtIO driver -> Mail Box Driver"传递数据到共享内存，实现 IPC 通信功能。

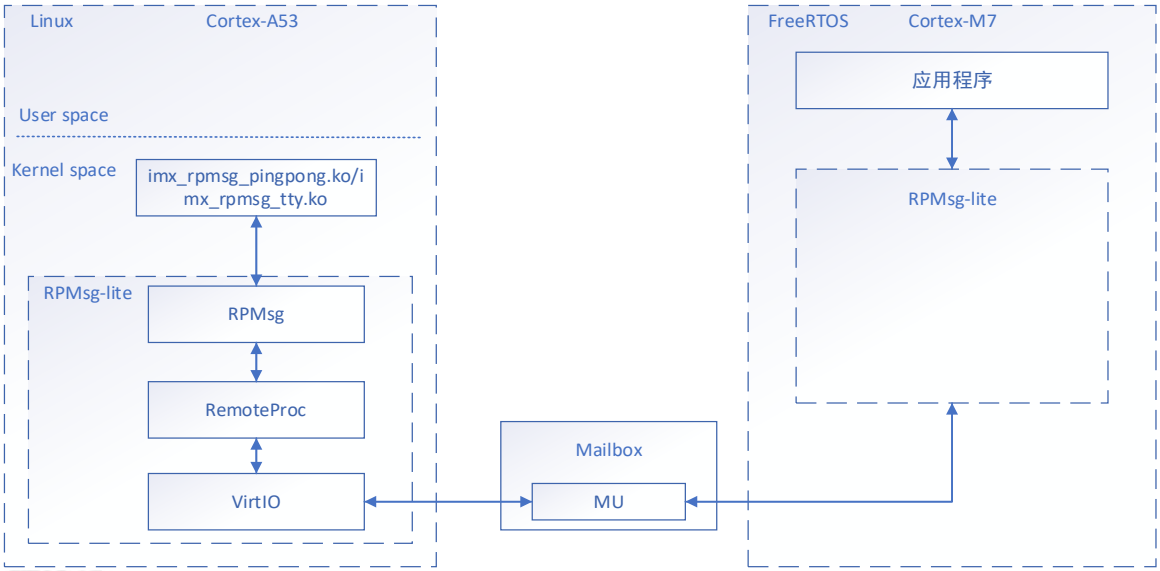

图 3

备注：

（1） 在 Linux 驱动层（内核空间）基于 imx_rpmsg_pingpong 或 imx_rpmsg_tty 驱动调用接口均可实现与 Cortex-M7 的核间通信。

（2） imx_rpmsg_tty 驱动对 RPMsg-lite 接口进行了封装，可能导致接口调用过程不够直观。因此推荐用户使用 imx_rpmsg_pingpong 驱动接口，其直接调用 RPMsg-lite 接口，能够更直接、高效地实现与 Cortex-M7 处理器核心之间的核间通信。可参考 rpmsg_lite_pingpong_rtos 案例。

1.2 Cortex-M7 核心程序加载启动流程

IMX8MP 启动并加载 Linux 系统的流程如下图所示，在 U-Boot 的启动阶段，Cortex-M7 核心程序是通过从 Linux 启动卡的 BOOT 分区中加载 Cortex-M7 核心镜像文件(m7_app.bin)至 TCM 或 DDR 内存区域中运行。

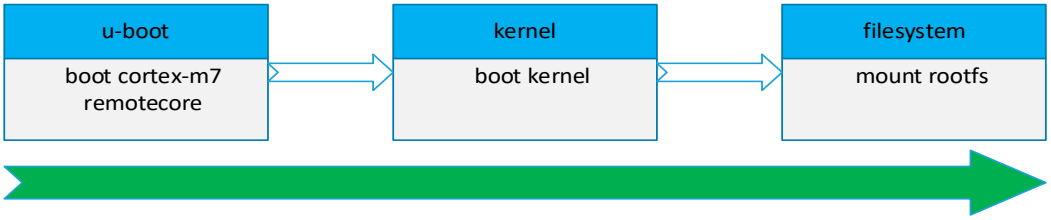

图 4

1.3 Linux 应用层空间与 Cortex-M7 核心通信

本章节以 rpmsg_lite_led_flash 案例为例，说明 Cortex-A53（主核心）和 Cortex-M7（远程核心）IPC 通信的函数调用流程，如下图所示。

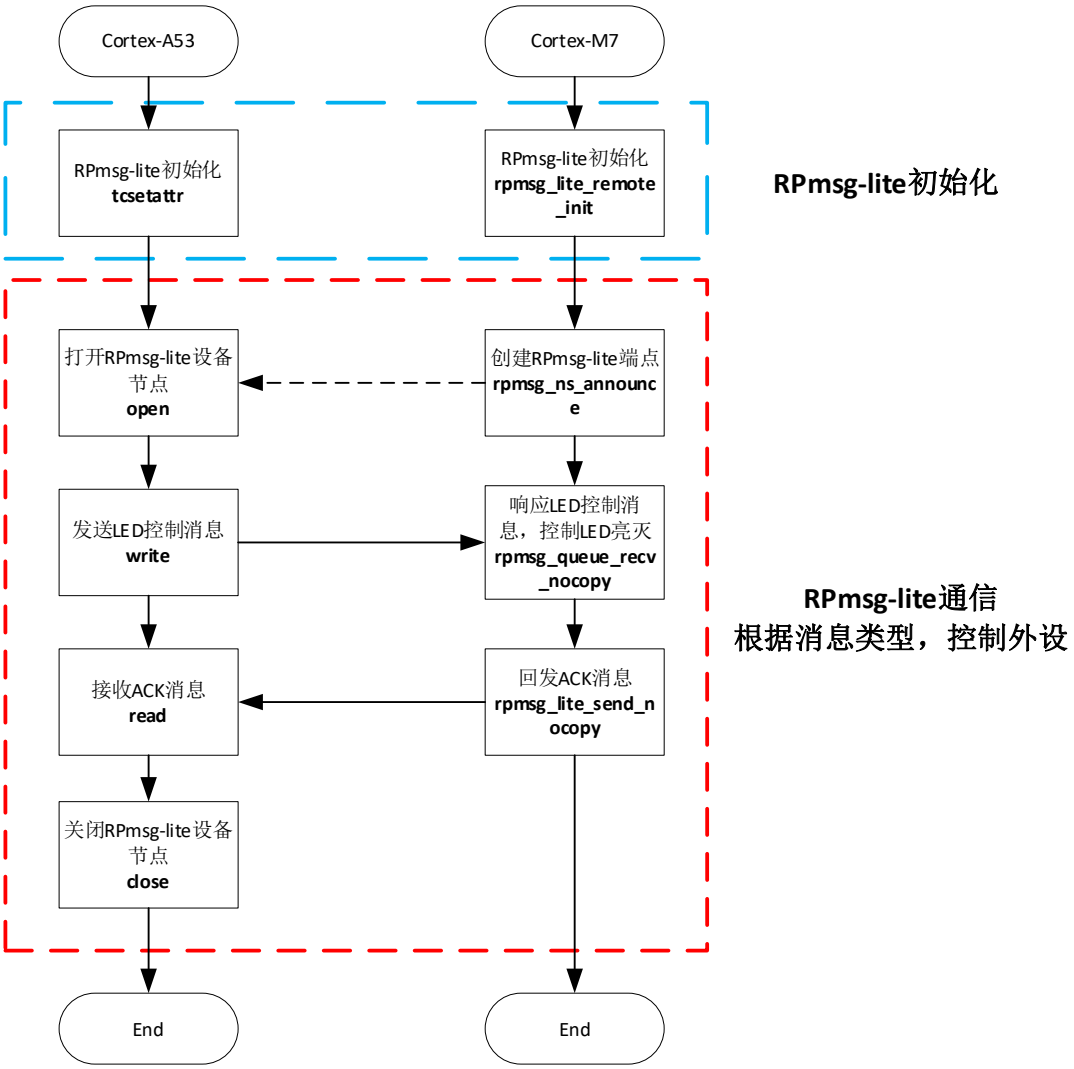

图 5

（1） Linux 端应用程序介绍

"/dev/ttyRPMsgxx"设备节点是由内核中的 imx_rpmsg_tty 驱动所注册的一个 TTY 终端设备，其专为应用层访问而设计。该驱动的核心功能是对 RPMsg-lite 接口进行封装，并将其抽象为一个虚拟串口设备。通过这种方式，用户在应用层只需使用标准的系统调用即可访问"/dev/ttyRPMsgxx"设备，实现与 Cortex-M7 核心的通信。这一机制简化了用户的操作，使他们无需深入了解底层 RPMsg-lite 接口的具体实现细节，用户仅需编写应用层的程序即可。

备注：Linux 应用层使用 RPMsg 进行核间通信，单次传输数据包大小最大为 256 字节，在驱动文件<内核源码目录>/drivers/rpmsg/imx_rpmsg_tty.c 中进行了相关定义。

```
/* this needs to be less then (RPMSG_BUF_SIZE - sizeof(struct rpmsg_hdr)) */
#define RPMSG_MAX_SIZE          256
```

图 6

（2） Linux 端常用 API 代码

下述为 Linux 端常用的 API 和功能介绍。

表 2

函数声明	函数作用
int open(const char *pathname,int flags,mode_t mode);	打开/dev/ttyRPMsgxx 虚拟串口设备
int close(int fd);	关闭/dev/ttyRPMsgxx 设备对应的文件描述符
int tcgetattr(int fd,struct termios *termios_p);	获取串口设备配置信息
int cfsetispeed(struct termios *termios_p,speed_t speed);	设置串口设备的输入波特率
int cfsetospeed(struct termios *termios_p,speed_t speed);	设置串口设备的输出波特率
ssize_t write(int fd, const void *buf,size_t count);	向串口设备写入数据
ssize_t read(int fd, void *buf,size_t count);	从串口设备读出数据
int tcsetattr(int fd,int optional_actions,const struct termios *termios_p);	设置串口设备配置信息

（3） Cortex-M7 端常用 API 代码

下述为 Cortex-M7 端常用的 RPMsg-lite API 和功能介绍。

表 3

函数声明	函数作用
struct rpmsg_lite_instance* rpmsg_lite_remote_init(void * shmem_addr,uint32_t link_id,uint32_t init_flags);	初始化 RPMsg 通信栈。必须在调用其他 RPMsg-Lite API 之前调用
rpmsg_queue_handle rpmsg_queue_create(struct rpmsg_lite_instance * rpmsg_lite_dev);	创建一个可用于阻塞接收的 RPMsg 队列
struct rpmsg_lite_endpoint* rpmsg_lite_create_ept(struct rpmsg_lite_instance * rpmsg_lite_dev,uint32_t addr,rl_ept_rx_cb_t rx_cb,void * rx_cb_data);	创建一个新的 RPMsg 端点，用于通信
int32_t rpmsg_ns_announce(struct rpmsg_lite_instance * rpmsg_lite_dev,struct rpmsg_lite_endpoint * new_ept,const char * ept_name,uint32_t flags);	发送名称服务通知到远程设备
int32_t rpmsg_queue_recv_nocopy(struct rpmsg_lite_instance * rpmsg_lite_dev,rpmsg_queue_handle q,uint32_t * src,char ** data,uint32_t * len,uintptr_t timeout);	阻塞接收数据
void* rpmsg_lite_alloc_tx_buffer(struct rpmsg_lite_instance * rpmsg_lite_dev,uint32_t * size,uintptr_t timeout);	为消息有效负载分配发送缓冲区
int32_t rpmsg_lite_send_nocopy(struct rpmsg_lite_instance * rpmsg_lite_dev,struct rpmsg_lite_endpoint * ept,uint32_t dst,void * data,uint32_t size);	在 rpmsg_lite_alloc_tx_buffer 分配的发送缓冲区中发送消息
int32_t rpmsg_queue_nocopy_free(struct rpmsg_lite_instance * rpmsg_lite_dev,void * data);	释放先前由 rpmsg_queue_recv_nocopy() 返回的缓冲区
int32_t rpmsg_lite_destroy_ept(struct rpmsg_lite_instance * rpmsg_lite_dev,struct rpmsg_lite_endpoint * rl_ept);	删除 RPMsg 端点并执行清理
uint32_t rpmsg_lite_wait_for_link_up(struct rpmsg_lite_instance * rpmsg_lite_dev,uint32_t timeout);	等待链路建立完成
int32_t rpmsg_queue_destroy(struct rpmsg_lite_instance * rpmsg_lite_dev,rpmsg_queue_handle q);	销毁队列并清理

2 openamp-demos

openamp-demos 主要演示 i.MX 8M Plus 的 Cortex-A53 核心和 Cortex-M7 核心的核间通信，其中 Cortex-A53 核心运行 Linux 系统，Cortex-M7 核心运行 FreeRTOS 系统。请先参考如下步骤替换支持 Cortex-M7 核心 RPMsg 案例的设备树文件，再进行 openamp-demos 案例的测试。

请将 Linux 系统启动卡 BOOT 分区中的支持 Cortex-M7 核心 RPMsg 案例的 tlimx8mp-evm-m7-rpmsg.dtb 设备树复制并重命名为 tlimx8mp-evm.dtb，替换分区中原来默认的 tlimx8mp-evm.dtb 设备树。

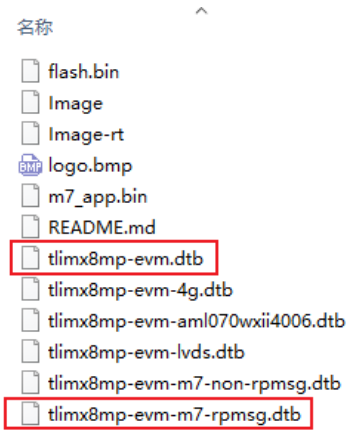

图 7

备注：设备树文件替换具体说明如下。

Cortex-M7 核心 RPMsg 设备树文件在基础设备树文件 tlimx8mp-evm.dts 的基础上增加了 Remote Processor(rproc)设备节点，主要用于实现 Cortex-M7 核心与 Cortex-A53 核心核间通信。

```
imx8mp-cm7 {
        compatible = "fsl,imx8mn-cm7";
        rsc-da = <0x55000000>;
        clocks = <&clk IMX8MP_CLK_M7_DIV>;
        mbox-names = "tx", "rx", "rxdb";
        mboxes = <&mu 0 1
                  &mu 1 1
                  &mu 3 1>;
        memory-region = <&vdevbuffer>, <&vdev0vring0>, <&vdev0vring1>, <&rsc_table>;
        status = "okay";
        fsl,startup-delay-ms = <500>;
};
```

图 8 tlimx8mp-evm-m7-rpmsg.dts

由于 Cortex-M7 核心内部默认不加载 Cortex-M7 核心程序，使能 Cortex-M7 核心程序未使用的外设将会与 Cortex-A53 核心发生冲突，因此 Cortex-M7 核心 RPMsg 设备树文件在基础设备树文件 tlimx8mp-evm.dts 的基础上将未使用的外设进行禁用。

```
&ecspi2 {
        status = "disabled";
};

&flexcan1 {
        status = "disabled";
};

&flexspi {
        status = "disabled";
};

&i2c3 {
        status = "disabled";
};

&pwm4{
        status = "disabled";
};

&sai3 {
        status = "disabled";
};

&micfil {
        status = "disabled";
};

&sdma3{
        status = "disabled";
};

&uart3 {
        status = "disabled";
};

&uart4 {
        status = "disabled";
};
```

图 9 tlimx8mp-evm-m7-rpmsg.dts

2.1 rpmsg_lite_led_flash 案例

2.1.1 案例说明

案例功能：本案例主要演示使用 RPMsg-lite 框架进行核间通信，实现 Cortex-A53 核心以 0.5 秒的时间间隔发送 RPMsg 数据包至 Cortex-M7 核心，点亮或熄灭评估板用户可编程指示灯。

系统框图和通信流程图如下图所示。

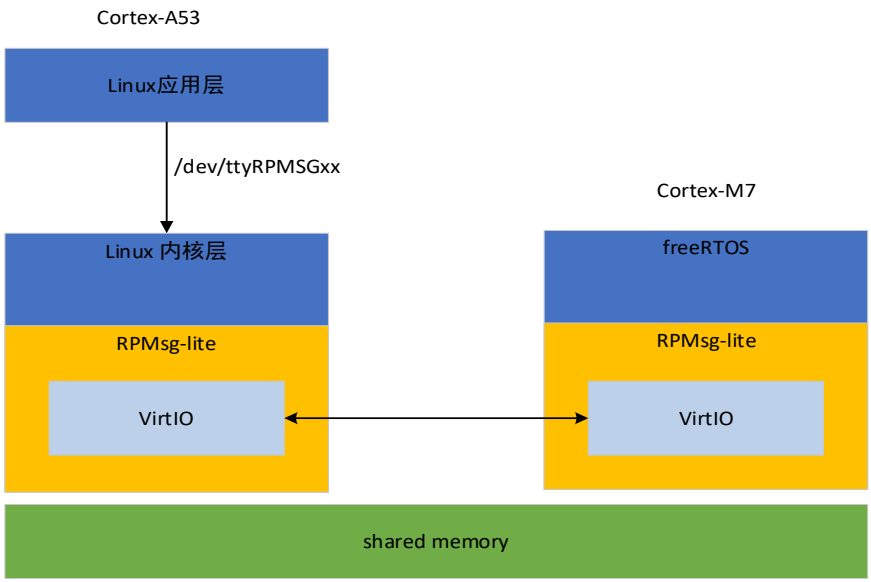

图 10 系统框图

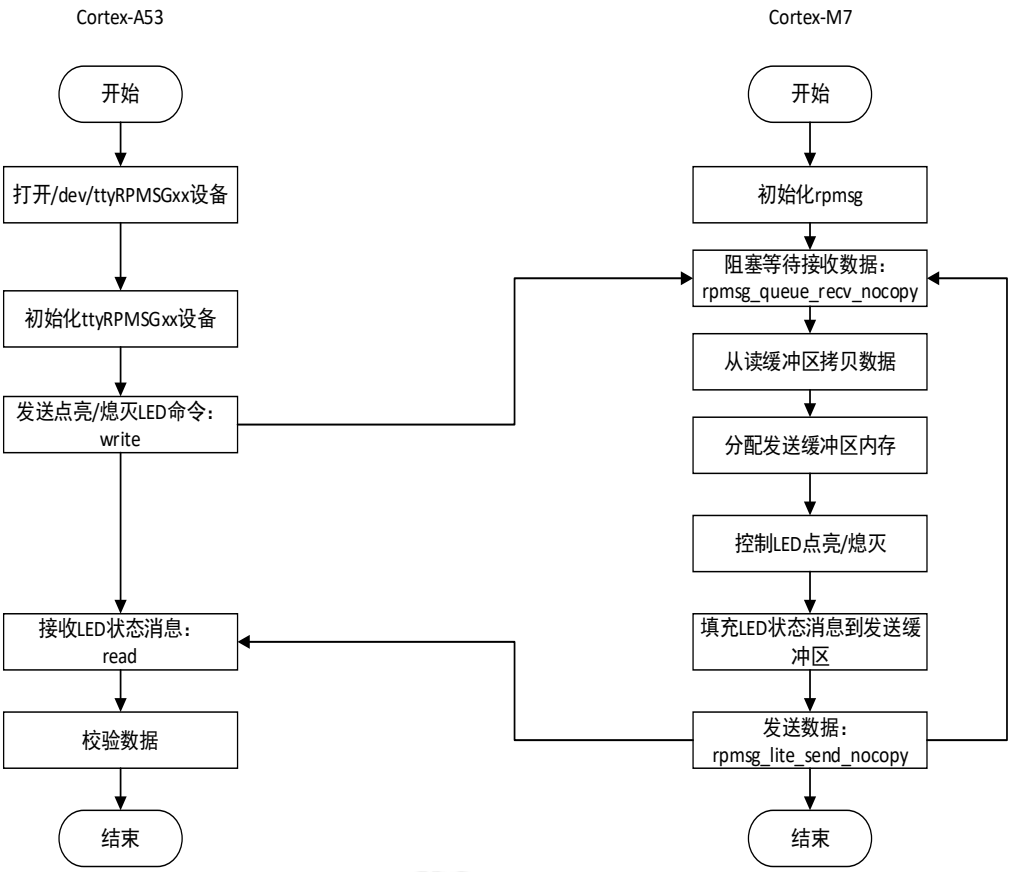

图 11 通信流程图

2.1.2 案例测试

（1） 加载运行 Cortex-M7 核心程序镜像文件

案例 bin 目录下分别提供了 TCM 版本程序镜像文件 rpmsg_lite_led_flash_rtos.bin 和 DDR 版本程序镜像文件 rpmsg_lite_led_flash_rtos_ddr.bin。请将 TCM 或 DDR 版本的 xxx.bin 程序镜像文件重命名为 m7_app.bin，并将其拷贝至 Linux 系统启动卡 BOOT 分区中。

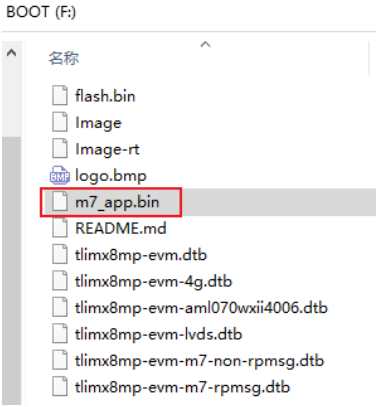

图 12

将 Linux 系统启动卡插至评估板 Micro SD 卡槽，启动方式选择拨码开关拨至 1100(1~4)，此档位为 Micro SD 卡启动模式。使用 USB 串口线将评估板的 USB TO UART2 串口连接至 PC 机的 USB 接口，使用 RS232 交叉串口母母线、USB 转 RS232 公头串口线将评估板的 RS232 UART4 串口连接至 PC 机的 USB 接口，如下图所示。

其中 USB TO UART2 作为 Cortex-A53 调试串口，RS232 UART4 作为 Cortex-M7 调试串口。

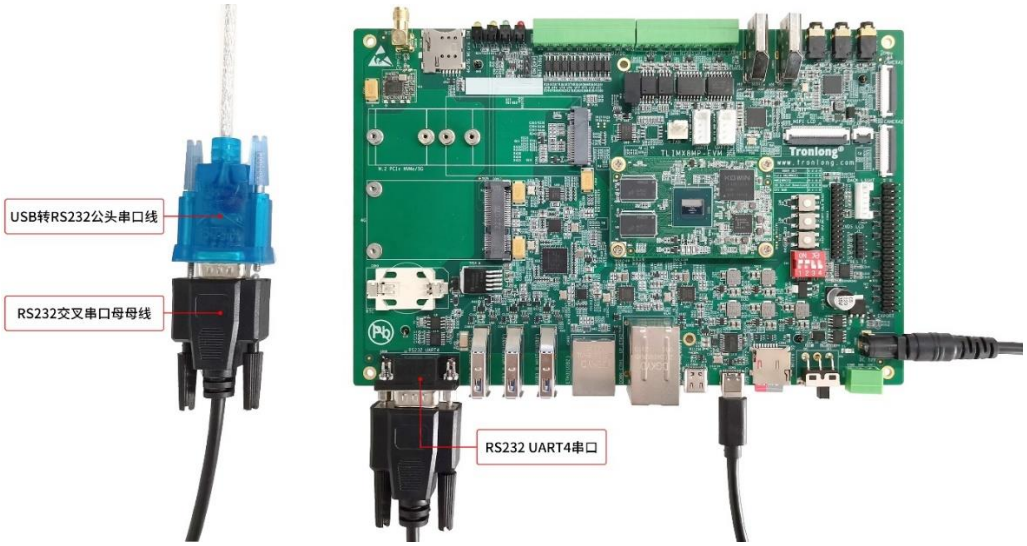

图 13

系统默认是 TCM 版本 Cortex-M7 核心程序镜像文件的运行环境，评估板上电启动，在 U-Boot 程序运行时，RS232 UART4 调试串口打印如下类似信息。

```
RPMSG LED FLASH FreeRTOS RTOS API Demo...
Nameservice sent, ready for incoming messages...
```

图 14

如需运行 DDR 版本程序镜像文件，请参考如下步骤进行设置，否则请忽略此内容。

评估板上电启动，在 U-Boot 倒计时结束之前按下空格键进入 U-Boot 命令行模式，在 U-Boot 命令行执行如下命令，设置 DDR 版本的 Cortex-M7 核心程序镜像文件运行环境。

U-Boot=> setenv m7_run_mem DDR

U-Boot=> saveenv

U-Boot=> reset

```
u-boot=>
u-boot=> setenv m7_run_mem DDR
u-boot=> saveenv
Saving Environment to MMC... Writing to MMC(2)... OK
u-boot=> reset
```

图 15

评估板重启，在 U-Boot 运行阶段，RS232 UART4 调试串口打印如下类似信息。

```
RPMSG LED FLASH FreeRTOS RTOS API Demo...
Nameservice sent, ready for incoming messages...
```

图 16

（2） 功能测试

进入评估板文件系统，执行如下命令加载 imx_rpmsg_tty.ko 驱动。

Target# modprobe imx_rpmsg_tty

```
root@IMX8MP-Tronlong:~# modprobe imx_rpmsg_tty
root@IMX8MP-Tronlong:~# [  449.024941] imx_rpmsg_tty virtio0.rpmsg-virtual-tty-channel-1.-1.30: new channel: 0x400 -> 0x1e!
[  449.025107] Install rpmsg tty driver!

root@IMX8MP-Tronlong:~#
```

图 17

同时 RS232 UART4 调试串口打印如下类似信息，表示驱动正常加载。

```
RPMSG LED FLASH FreeRTOS RTOS API Demo...

Nameservice sent, ready for incoming messages...
Get Message From Master Side : "hello world!" [len : 12]
Non-led control commands:hello world! ......
```

图 18

驱动成功加载后，执行如下命令确认生成"/dev/ttyRPMSGxx"设备。

Target# ls /dev/ttyRPMSG*

```
root@IMX8MP-Tronlong:~# ls /dev/ttyRPMSG*
/dev/ttyRPMSG30
root@IMX8MP-Tronlong:~#
```

图 19

请将案例 bin 目录下的 rpmsg_lite_led_flash 文件拷贝至评估板文件系统，执行如下命令，查看程序相关参数。

Target# ./rpmsg_lite_led_flash -h

```
root@IMX8MP-Tronlong:~# ls rpmsg_lite_led_flash
rpmsg_lite_led_flash
root@IMX8MP-Tronlong:~# ./rpmsg_lite_led_flash -h
Usage: rpmsg_lite_led_flash [options]

Options:
 -d | --device        Device
 -h | --help          Show help content

Example (single):
  # ./rpmsg_lite_led_flash -d /dev/ttyRPMSG30
root@IMX8MP-Tronlong:~#
```

图 20

参数解析：

-d：设备节点。

-h：打印帮助信息。

执行如下命令，控制用户可编程指示灯 LED1 以 0.5 秒时间间隔闪烁 10 次，调试串口终端打印信息如下图所示。

Target# ./rpmsg_lite_led_flash -d /dev/ttyRPMSG30

```
root@IMX8MP-Tronlong:~# ./rpmsg_lite_led_flash -d /dev/ttyRPMSG30
send data size: 6,send data: LED ON
read data size: 6,read data: SET OK
send data size: 7,send data: LED OFF
read data size: 6,read data: SET OK
send data size: 6,send data: LED ON
read data size: 6,read data: SET OK
send data size: 7,send data: LED OFF
read data size: 6,read data: SET OK
send data size: 6,send data: LED ON
read data size: 6,read data: SET OK
send data size: 7,send data: LED OFF
read data size: 6,read data: SET OK
send data size: 6,send data: LED ON
read data size: 6,read data: SET OK
send data size: 7,send data: LED OFF
read data size: 6,read data: SET OK
send data size: 6,send data: LED ON
read data size: 6,read data: SET OK
send data size: 7,send data: LED OFF
read data size: 6,read data: SET OK
send data size: 6,send data: LED ON
read data size: 6,read data: SET OK
send data size: 7,send data: LED OFF
read data size: 6,read data: SET OK
send data size: 6,send data: LED ON
read data size: 6,read data: SET OK
send data size: 7,send data: LED OFF
read data size: 6,read data: SET OK
send data size: 6,send data: LED ON
read data size: 6,read data: SET OK
send data size: 7,send data: LED OFF
read data size: 6,read data: SET OK
send data size: 6,send data: LED ON
read data size: 6,read data: SET OK
send data size: 7,send data: LED OFF
read data size: 6,read data: SET OK
Successful test .....
root@IMX8MP-Tronlong:~#
```

图 21

同时 RS232 UART4 调试串口打印如下类似信息。

```
RPMSG LED FLASH FreeRTOS RTOS API Demo...

Nameservice sent, ready for incoming messages...
Get Message From Master Side : "hello world!" [len : 12]
Non-led control commands:hello world! ......
Get Message From Master Side : "LED ON" [len : 6]
Get Message From Master Side : "LED OFF" [len : 7]
Get Message From Master Side : "LED ON" [len : 6]
Get Message From Master Side : "LED OFF" [len : 7]
Get Message From Master Side : "LED ON" [len : 6]
Get Message From Master Side : "LED OFF" [len : 7]
Get Message From Master Side : "LED ON" [len : 6]
Get Message From Master Side : "LED OFF" [len : 7]
Get Message From Master Side : "LED ON" [len : 6]
Get Message From Master Side : "LED OFF" [len : 7]
Get Message From Master Side : "LED ON" [len : 6]
Get Message From Master Side : "LED OFF" [len : 7]
Get Message From Master Side : "LED ON" [len : 6]
Get Message From Master Side : "LED OFF" [len : 7]
Get Message From Master Side : "LED ON" [len : 6]
Get Message From Master Side : "LED OFF" [len : 7]
Get Message From Master Side : "LED ON" [len : 6]
Get Message From Master Side : "LED OFF" [len : 7]
Get Message From Master Side : "LED ON" [len : 6]
Get Message From Master Side : "LED OFF" [len : 7]
```

图 22

2.1.3 案例编译

（1） RTOS 程序编译

请根据《Cortex-M7 核心开发手册》文档的“Cortex-M7 核心案例编译”章节将案例 project 目录下的程序源码进行编译，生成程序镜像文件。

（2） Linux 程序编译

请将产品资料“4-软件资料\Demo\openamp-demos\rpmsg_lite_led_flash\src\”下的文件拷贝至 Ubuntu 工作目录。

进入案例应用程序源码目录，执行如下命令进行编译，编译完成将会在当前目录下生成可执行文件。

Host# source /home/tronlong/SDK/environment-setup-armv8a-poky-linux

Host# make

```
tronlong@tronlong-virtual-machine:~/Demo/openamp-demos/rpmsg_lite_led_flash/src$ ls
Makefile  rpmsg_lite_led_flash.c
tronlong@tronlong-virtual-machine:~/Demo/openamp-demos/rpmsg_lite_led_flash/src$ source /home/tronlong/
SDK/environment-setup-armv8a-poky-linux
tronlong@tronlong-virtual-machine:~/Demo/openamp-demos/rpmsg_lite_led_flash/src$ make
aarch64-poky-linux-gcc  -march=armv8-a+crc+crypto -fstack-protector-strong  -O2 -D_FORTIFY_SOURCE=2 -Wf
ormat -Wformat-security -Werror=format-security --sysroot=/home/tronlong/SDK/sysroots/armv8a-poky-linux
 -Wall -O2 rpmsg_lite_led_flash.c -o rpmsg_lite_led_flash
tronlong@tronlong-virtual-machine:~/Demo/openamp-demos/rpmsg_lite_led_flash/src$ ls
Makefile  rpmsg_lite_led_flash  rpmsg_lite_led_flash.c
tronlong@tronlong-virtual-machine:~/Demo/openamp-demos/rpmsg_lite_led_flash/src$
```

图 23

2.1.4 关键代码

Cortex-M7 核心程序

（1） 初始化 Cortex-M7 核心和 GPIO。

```
/* Initialize standard SDK demo application pins */
/* M7 has its local cache and enabled by default,
 * need to set smart subsystems (0x28000000 ~ 0x3FFFFFFF)
 * non-cacheable before accessing this address region */
BOARD_InitMemory();

/* Board specific RDC settings */
BOARD_RdcInit();

BOARD_InitBootPins();
BOARD_BootClockRUN();
BOARD_InitDebugConsole();

copyResourceTable();

/* Init output LED GPIO. */
GPIO_PinInit(EXAMPLE_LED_GPIO, EXAMPLE_LED_GPIO_PIN, &led_config);
```

图 24

（2） 创建并调度任务。

```
/* Create the led_flash task. */
if (xTaskCreate(led_flash_task, "led_flash_task", LDE_FLASH_TASK_STACK_SIZE, NULL, tskIDLE_PRIORITY + 1, &led_flash_task_handle) != pdPASS)
{
    PRINTF("\r\nFailed to create application task\r\n");
    for (;;)
        ;
}
vTaskStartScheduler();
```

图 25

（3） 初始化 RPMsg-lite，进行数据收发。

```
led_flash_rpmsg = rpmsg_lite_remote_init((void *)RPMSG_LITE_SHMEM_BASE, RPMSG_LITE_LINK_ID, RL_NO_FLAGS);

rpmsg_lite_wait_for_link_up(led_flash_rpmsg);

led_flash_queue = rpmsg_queue_create(led_flash_rpmsg);
led_flash_ept   = rpmsg_lite_create_ept(led_flash_rpmsg, LOCAL_EPT_ADDR, rpmsg_queue_rx_cb, led_flash_queue);
(void)rpmsg_ns_announce(led_flash_rpmsg, led_flash_ept, RPMSG_LITE_NS_ANNOUNCE_STRING, RL_NS_CREATE);
```

图 26

（4） 接收 Cortex-A53 核心发送的数据，将数据拷贝至缓冲区并分配发送缓冲区内存。

```
/* Get RPMsg rx buffer with message */
result = rpmsg_queue_recv_nocopy(led_flash_rpmsg, led_flash_queue, (uint32_t *)&remote_addr, (char **)&rx_buf, &len, RL_BLOCK);
if (result != 0)
{
    assert(false);
}

/* Copy string from RPMsg rx buffer */
assert(len < sizeof(app_buf));
memcpy(app_buf, rx_buf, len);
app_buf[len] = 0; /* End string by '\0' */

if ((len == 2) && (app_buf[0] == 0xd) && (app_buf[1] == 0xa))
    PRINTF("Get New Line From Master Side\r\n");
else
    PRINTF("Get Message From Master Side : \"%s\" [len : %d]\r\n", app_buf, len);

/* Get tx buffer from RPMsg */
tx_buf = rpmsg_lite_alloc_tx_buffer(led_flash_rpmsg, &size, RL_BLOCK);
assert(tx_buf);
```

图 27

（5） LED 亮灭控制，并将 LED 状态消息发送至 Cortex-A53 核心。

```
        /* Gpio output height level, turn on the LED */
        GPIO_PinWrite(EXAMPLE_LED_GPIO, EXAMPLE_LED_GPIO_PIN, 1U);

        /* Copy string to RPMsg tx buffer */
        memcpy(tx_buf, LED_SET_STATUS, strlen(LED_SET_STATUS));
        /* Echo back received message with nocopy send */
        result = rpmsg_lite_send_nocopy(led_flash_rpmsg, led_flash_ept, remote_addr, tx_buf, strlen(LED_SET_STATUS));
        if (result != 0)
        {
            assert(false);
        }

        /* Keep the LED on at least for 500 ms */
        SDK_DelayAtLeastUs(500000, SDK_DEVICE_MAXIMUM_CPU_CLOCK_FREQUENCY);

    }else if(strncmp("LED OFF", app_buf, len) == 0){
        /* Keep the LED off at least for 500 ms */
        GPIO_PinWrite(EXAMPLE_LED_GPIO, EXAMPLE_LED_GPIO_PIN, 0U);

        /* Copy string to RPMsg tx buffer */
        memcpy(tx_buf, LED_SET_STATUS, strlen(LED_SET_STATUS));
        /* Echo back received message with nocopy send */
        result = rpmsg_lite_send_nocopy(led_flash_rpmsg, led_flash_ept, remote_addr, tx_buf, strlen(LED_SET_STATUS));
        if (result != 0)
        {
            assert(false);
        }
```

图 28

（6） 释放资源。

```
/* Release held RPMsg rx buffer */
result = rpmsg_queue_nocopy_free(led_flash_rpmsg, rx_buf);
if (result != 0)
{
    assert(false);
}
```

图 29

Linux 应用程序

（1） 打开"/dev/ttyRPMSG30"设备节点。

```
    if((fd = open(dev, O_RDWR | O_NOCTTY )) < 0)
    {
        printf("open %s is error.\n",argv[1]);
        return -1;
    }
```

图 30

（2） 初始化 ttyRPMSG30 设备。

```
    /* /dev/ttyRPMSG30 init */
    tcgetattr(fd, &opt);
    cfsetispeed(&opt, B115200);
    cfsetospeed(&opt, B115200);
    tcflush(fd, TCIOFLUSH);

    opt.c_cflag |= (CLOCAL | CREAD);
    opt.c_cflag &= ~CSIZE;
    opt.c_cflag &= ~CRTSCTS;
    opt.c_cflag |= CS8;
    opt.c_iflag |= IGNPAR;
    opt.c_cflag &= ~CSTOPB;
    opt.c_oflag = 0;
    opt.c_lflag = 0;

    tcsetattr(fd, TCSANOW, &opt);
```

图 31

（3） 发送 LED 控制命令，读取 LED 状态消息。

```
    while(i<10){
        memset(send_buf, 0, sizeof(send_buf));
        strcpy(send_buf,"LED ON");

        /* open led */
        count = write(fd, send_buf, strlen(send_buf));
        printf("send data size: %d,send data: %s\n", count, send_buf);
        sleep(0.5);
        count = read(fd, read_buf, sizeof(read_buf));
        printf("read data size: %d,read data: %s\n", count, read_buf);

        memset(send_buf, 0, sizeof(send_buf));
        strcpy(send_buf,"LED OFF");

    sleep(1);

        /* close led */
        count = write(fd, send_buf, strlen(send_buf));
        printf("send data size: %d,send data: %s\n", count, send_buf);
        sleep(0.5);
        count = read(fd, read_buf, sizeof(read_buf));
        printf("read data size: %d,read data: %s\n", count, read_buf);

        i++;
    }
```

图 32

2.2 rpmsg_lite_pingpong_rtos 案例

2.2.1 案例说明

案例功能：本案例基于 RPMsg-lite 框架进行核间通信。实现 Cortex-A53 核心在内核层通过 RPMsg-lite 框架发送 ping 消息，Cortex-M7 核心接收到 ping 消息后向 Cortex-A53 核心发送 pong 消息，pingpong 消息次数为 100 次。

程序原理流程图如下所示：

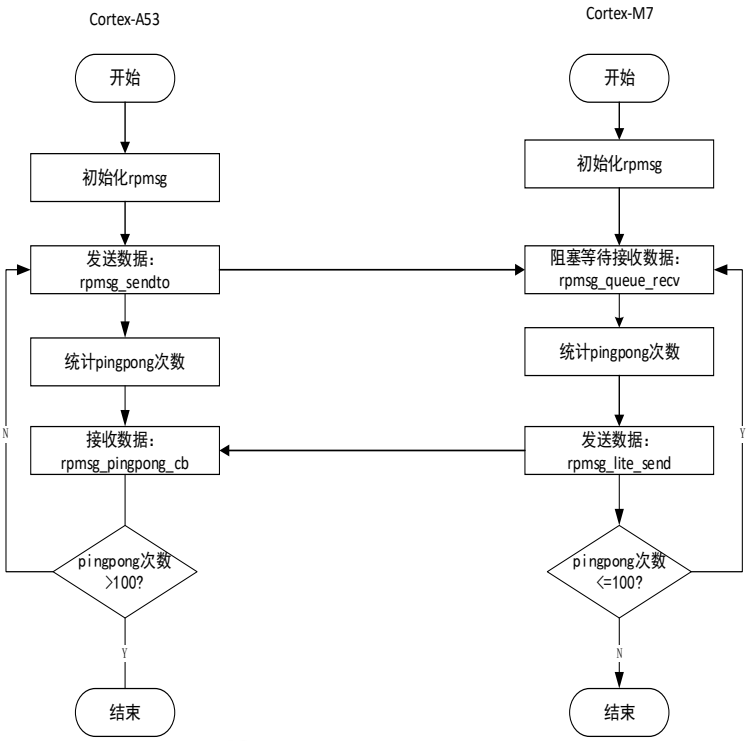

图 33

i.MX8MP 的 Cortex-A53 核心与 Cortex-M7 核心基于 RPMsg-lite 框架实现核间通信。本案例的 Cortex-A53 核心通过加载 imx_rpmsg_pingpong 驱动，在内核驱动中直接调用 RPMsg-lite 接口实现与 Cortex-M7 核心通信。

系统框图如下图所示。

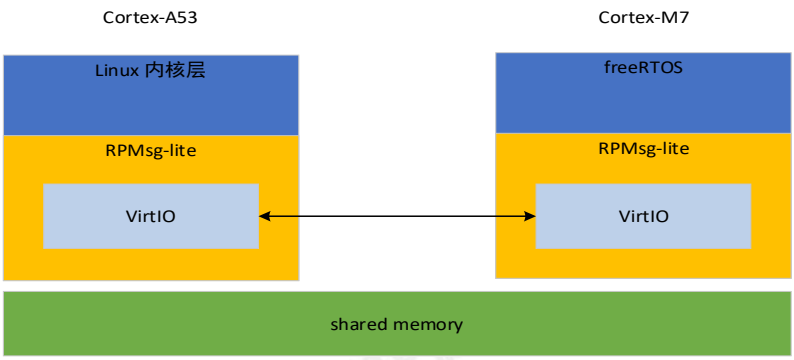

图 34 系统框图

2.2.2 案例测试

（1） 加载运行 Cortex-M7 核心程序镜像文件

案例 bin 目录下分别提供了 TCM 版本程序镜像文件 rpmsg_lite_pingpong_rtos.bin 和 DDR 版本程序镜像文件 rpmsg_lite_pingpong_rtos_ddr.bin。请将 TCM 或 DDR 版本的 xxx.bin 程序镜像文件重命名为 m7_app.bin，并将其拷贝至 Linux 系统启动卡 BOOT 分区中。

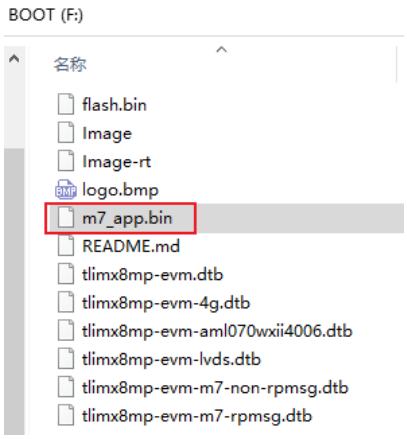

图 35

将 Linux 系统启动卡插至评估板 Micro SD 卡槽，启动方式选择拨码开关拨至 1100(1~4)，此档位为 Micro SD 卡启动模式。使用 USB 串口线将评估板的 USB TO UART2 串口连接至 PC 机的 USB 接口，使用 RS232 交叉串口母母线、USB 转 RS232 公头串口线将评估板的 RS232 UART4 串口连接至 PC 机的 USB 接口，如下图所示。

其中 USB TO UART2 作为 Cortex-A53 核心调试串口，RS232 UART4 作为 Cortex-M7 核心调试串口。

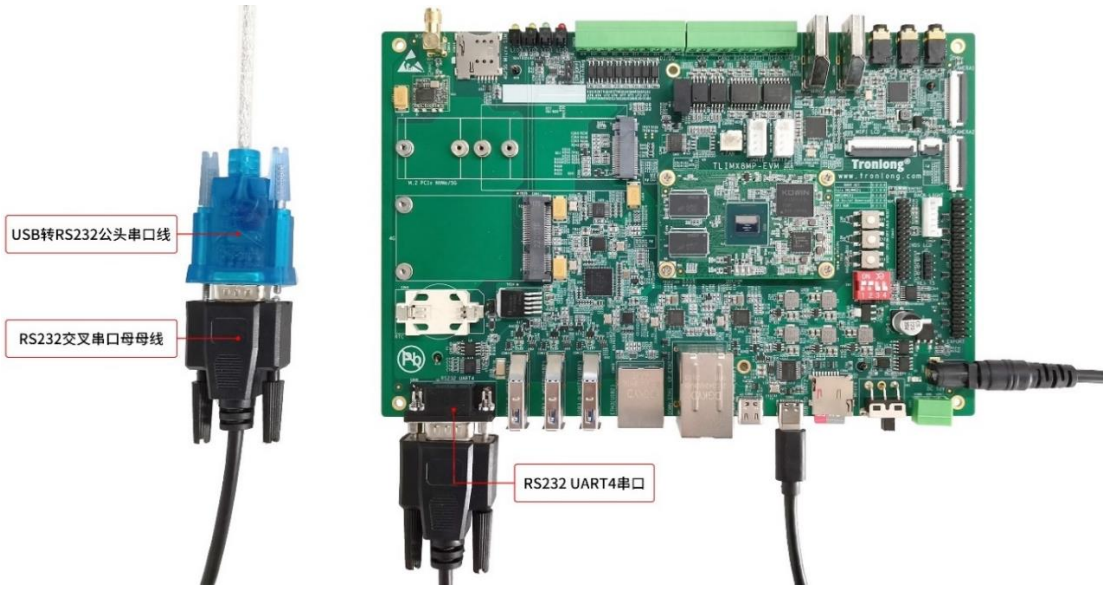

图 36

系统默认是 TCM 版本 Cortex-M7 核心程序镜像文件的运行环境，评估板上电启动，在 U-Boot 程序运行时，RS232 UART4 调试串口打印如下类似信息。

```
RPMSG Ping-Pong FreeRTOS RTOS API Demo ...
RPMSG Share Base Addr is 0x55000000
Link is up!
Nameservice announce sent.
```

图 37

如需运行 DDR 版本程序镜像文件，请参考如下步骤进行设置，否则请忽略此内容。

评估板上电启动，在 U-Boot 倒计时结束之前按下空格键进入 U-Boot 命令行模式，在 U-Boot 命令行执行如下命令，设置 DDR 版本的 Cortex-M7 核心程序镜像文件运行环境。

U-Boot=> setenv m7_run_mem DDR

U-Boot=> saveenv

U-Boot=> reset

```
u-boot=>
u-boot=> setenv m7_run_mem DDR
u-boot=> saveenv
Saving Environment to MMC... Writing to MMC(2)... OK
u-boot=> reset
```

图 38

评估板重启，在 U-Boot 程序运行时，RS232 UART4 调试串口打印如下类似信息。

```
RPMSG Ping-Pong FreeRTOS RTOS API Demo ...
RPMSG Share Base Addr is 0x55000000
Link is up!
Nameservice announce sent.
```

图 39

进入评估板文件系统后，执行如下命令加载 imx_rpmsg_pingpong 模块，启动核间通信。

Target# modprobe imx_rpmsg_pingpong

```
root@IMX8MP-Tronlong:~# modprobe imx_rpmsg_pingpong
[  109.683795] imx_rpmsg_pingpong virtio0.rpmsg-openamp-demo-channel.-1.30: new channel: 0x400 -> 0x1e!
root@IMX8MP-Tronlong:~# [  109.693431] get 1 (src: 0x1e)
[  109.694957] get 3 (src: 0x1e)
[  109.698284] get 5 (src: 0x1e)
[  109.701605] get 7 (src: 0x1e)
[  109.704927] get 9 (src: 0x1e)
[  109.708251] get 11 (src: 0x1e)
[  109.711566] get 13 (src: 0x1e)
[  109.714892] get 15 (src: 0x1e)
[  109.718216] get 17 (src: 0x1e)
[  109.721536] get 19 (src: 0x1e)
[  109.724859] get 21 (src: 0x1e)
[  109.728186] get 23 (src: 0x1e)
[  109.731501] get 25 (src: 0x1e)
[  109.734827] get 27 (src: 0x1e)
[  109.738151] get 29 (src: 0x1e)
[  109.741478] get 31 (src: 0x1e)
[  109.744791] get 33 (src: 0x1e)
[  109.748116] get 35 (src: 0x1e)
[  109.751437] get 37 (src: 0x1e)
[  109.755081] get 39 (src: 0x1e)
[  109.758088] get 41 (src: 0x1e)
[  109.761409] get 43 (src: 0x1e)
[  109.764733] get 45 (src: 0x1e)
[  109.768213] get 47 (src: 0x1e)
[  109.771370] get 49 (src: 0x1e)
[  109.774699] get 51 (src: 0x1e)
[  109.778017] get 53 (src: 0x1e)
[  109.781344] get 55 (src: 0x1e)
[  109.784665] get 57 (src: 0x1e)
[  109.787980] get 59 (src: 0x1e)
[  109.791307] get 61 (src: 0x1e)
[  109.794632] get 63 (src: 0x1e)
[  109.797953] get 65 (src: 0x1e)
[  109.801274] get 67 (src: 0x1e)
[  109.804612] get 69 (src: 0x1e)
[  109.807912] get 71 (src: 0x1e)
[  109.811239] get 73 (src: 0x1e)
[  109.814564] get 75 (src: 0x1e)
[  109.818185] get 77 (src: 0x1e)
[  109.821507] get 79 (src: 0x1e)
[  109.824530] get 81 (src: 0x1e)
[  109.827846] get 83 (src: 0x1e)
[  109.831178] get 85 (src: 0x1e)
[  109.834495] get 87 (src: 0x1e)
[  109.838108] get 89 (src: 0x1e)
[  109.841433] get 91 (src: 0x1e)
[  109.844463] get 93 (src: 0x1e)
[  109.847779] get 95 (src: 0x1e)
[  109.851404] get 97 (src: 0x1e)
[  109.854426] get 99 (src: 0x1e)
[  109.857752] get 101 (src: 0x1e)
[  109.857765] imx_rpmsg_pingpong virtio0.rpmsg-openamp-demo-channel.-1.30: goodbye!
```

图 40

加载模块后，Cortex-M7 将接收 Cortex-A53 发送的数据，并将其转发回 Cortex-A53，循环 50 次后结束。RS232 UART4 调试串口打印信息如下。

```
RPMSG Ping-Pong FreeRTOS RTOS API Demo ...
RPMSG Share Base Addr is 0x55000000
Link is up!
Nameservice announce sent.
Waiting for ping...
Sending pong...
Waiting for ping...
Sending pong...
Waiting for ping...
Sending pong...
Waiting for ping...
Sending pong...
Waiting for ping...
Sending pong...
Waiting for ping...
Sending pong...
Waiting for ping...
Sending pong...
```

图 41

```
Waiting for ping...
Sending pong...
Waiting for ping...
Sending pong...
Waiting for ping...
Sending pong...
Waiting for ping...
Sending pong...
Waiting for ping...
Sending pong...
Waiting for ping...
Sending pong...
Waiting for ping...
Sending pong...
Waiting for ping...
Sending pong...
Waiting for ping...
Sending pong...
Waiting for ping...
Sending pong...
Waiting for ping...
Sending pong...
Ping pong done, deinitializing...
Looping forever...
```

图 42

2.2.3 案例编译

请根据《Cortex-M7 核心开发手册》文档的“Cortex-M7 核心案例编译”章节将案例 project 目录下的程序源码进行编译，生成程序镜像文件。

2.2.4 关键代码

Linux 驱动程序

驱动的实现在 imx_rpmsg_pingpong.c 中完成，此文件位于内核源码"drivers/rpmsg/"路径下。

（1） 注册 rpmsg 驱动至内核。

```
static void rpmsg_pingpong_remove(struct rpmsg_device *rpdev)
{
        dev_info(&rpdev->dev, "rpmsg pingpong driver is removed\n");
}

static struct rpmsg_device_id rpmsg_driver_pingpong_id_table[] = {
        { .name = "rpmsg-openamp-demo-channel" },
        { .name = "rpmsg-openamp-demo-channel-1" },
        { },
};

static struct rpmsg_driver rpmsg_pingpong_driver = {
        .drv.name       = KBUILD_MODNAME,
        .drv.owner      = THIS_MODULE,
        .id_table       = rpmsg_driver_pingpong_id_table,
        .probe          = rpmsg_pingpong_probe,
        .callback       = rpmsg_pingpong_cb,
        .remove         = rpmsg_pingpong_remove,
};

```

图 43

```
static int __init init(void)
{
        return register_rpmsg_driver(&rpmsg_pingpong_driver);
}

static void __exit fini(void)
{
        unregister_rpmsg_driver(&rpmsg_pingpong_driver);
}
module_init(init);
module_exit(fini);

MODULE_AUTHOR("Freescale Semiconductor, Inc.");
MODULE_DESCRIPTION("iMX virtio remote processor messaging pingpong driver");
MODULE_LICENSE("GPL v2");
```

图 44

（2） Linux 系统加载驱动后，发送第一条乒乓信息。

```
static int rpmsg_pingpong_probe(struct rpmsg_device *rpdev)
{
        int err;
        unsigned int rpmsg_pingpong;

        dev_info(&rpdev->dev, "new channel: 0x%x -> 0x%x!\n",
                        rpdev->src, rpdev->dst);

        /*
         * send a message to our remote processor, and tell remote
         * processor about this channel
         */
        err = rpmsg_send(rpdev->ept, MSG, strlen(MSG));
        if (err) {
                dev_err(&rpdev->dev, "rpmsg_send failed: %d\n", err);
                return err;
        }

        rpmsg_pingpong = 0;
        err = rpmsg_sendto(rpdev->ept, (void *)(&rpmsg_pingpong),
                           4, rpdev->dst);
        if (err) {
                dev_err(&rpdev->dev, "rpmsg_send failed: %d\n", err);
                return err;
        }

        return 0;
}
```

图 45

（3） 在回调函数中，每次接收到 Cortex-M7 的计数器数据后自加 1，并回发至 Cortex-M7，直到计数器数值大于 100。

```
static int rpmsg_pingpong_cb(struct rpmsg_device *rpdev, void *data, int len,
                                                void *priv, u32 src)
{
        int err;
        unsigned int rpmsg_pingpong;

        /* reply */
        rpmsg_pingpong = *(unsigned int *)data;
        pr_info("get %d (src: 0x%x)\n", rpmsg_pingpong, src);

        /* pingpongs should not live forever */
        if (rpmsg_pingpong > 100) {
                dev_info(&rpdev->dev, "goodbye!\n");
                return 0;
        }
        rpmsg_pingpong++;
        err = rpmsg_sendto(rpdev->ept, (void *)(&rpmsg_pingpong), 4, src);

        if (err)
                dev_err(&rpdev->dev, "rpmsg_send failed: %d\n", err);

        return err;
}
```

图 46

Cortex-M7 核心程序

程序主体位于 main 函数中，定义位于 rpmsg_lite_pingpong_rtos.c 中。

（1） 在 main 函数中首先初始化相关资源，然后创建 app_task 任务，并启动任务列表。

```
    /* Initialize standard SDK demo application pins */
    /* M7 has its local cache and enabled by default,
     * need to set smart subsystems (0x28000000 ~ 0x3FFFFFFF)
     * non-cacheable before accessing this address region */
    BOARD_InitMemory();

    /* Board specific RDC settings */
    BOARD_RdcInit();

    BOARD_InitBootPins();
    BOARD_BootClockRUN();
    BOARD_InitDebugConsole();

    copyResourceTable();

#ifdef MCMGR_USED
    /* Initialize MCMGR before calling its API */
    (void)MCMGR_Init();
#endif /* MCMGR_USED */

    app_create_task();
    vTaskStartScheduler();

    (void)PRINTF("Failed to start FreeRTOS on core0.\r\n");
    for (;;)
    {
    }
```

图 47

（2） 在 app_create_task 任务中创建 app_task 任务与 Cortex-A53 核心进行通信。

```
void app_create_task(void)
{
    if (xTaskCreate(app_task, "APP_TASK", APP_TASK_STACK_SIZE, NULL, tskIDLE_PRIORITY + 1, &app_task_handle) != pdPASS)
    {
        PRINTF("\r\nFailed to create application task\r\n");
        for (;;)
            ;
    }
}
```

图 48

（3） 在 app_task 任务中初始化 RPMsg-lite 资源，并等待连接上远程终端进行数据收发。

```c
static void app_task(void *param)
{
    volatile uint32_t remote_addr;
    volatile rpmsg_ns_handle ns_handle;

    /* Print the initial banner */
    (void)PRINTF("\r\nRPMSG Ping-Pong FreeRTOS RTOS API Demo ...\r\n");

#ifdef MCMGR_USED
    uint32_t startupData;
    mcmgr_status_t status;

    /* Get the startup data */
    do
    {
        status = MCMGR_GetStartupData(&startupData);
    } while (status != kStatus_MCMGR_Success);

    my_rpmsg = rpmsg_lite_remote_init((void *)(char *)startupData, RPMSG_LITE_LINK_ID, RL_NO_FLAGS);

    /* Signal the other core we are ready by triggering the event and passing the APP_RPMSG_READY_EVENT_DATA */
    (void)MCMGR_TriggerEvent(kMCMGR_RemoteApplicationEvent, APP_RPMSG_READY_EVENT_DATA);
#else
    (void)PRINTF("RPMSG Share Base Addr is 0x%x\r\n", RPMSG_LITE_SHMEM_BASE);
    my_rpmsg = rpmsg_lite_remote_init((void *)RPMSG_LITE_SHMEM_BASE, RPMSG_LITE_LINK_ID, RL_NO_FLAGS);
#endif /* MCMGR_USED */
    rpmsg_lite_wait_for_link_up(my_rpmsg);
    (void)PRINTF("Link is up!\r\n");

    my_queue  = rpmsg_queue_create(my_rpmsg);
    my_ept    = rpmsg_lite_create_ept(my_rpmsg, LOCAL_EPT_ADDR, rpmsg_queue_rx_cb, my_queue);
    ns_handle = rpmsg_ns_bind(my_rpmsg, app_nameservice_isr_cb, ((void *)0));
    /* Introduce some delay to avoid NS announce message not being captured by the master side.
       This could happen when the remote side execution is too fast and the NS announce message is triggered
       before the nameservice_isr_cb is registered on the master side. */
    SDK_DelayAtLeastUs(1000000U, SDK_DEVICE_MAXIMUM_CPU_CLOCK_FREQUENCY);
    (void)rpmsg_ns_announce(my_rpmsg, my_ept, RPMSG_LITE_NS_ANNOUNCE_STRING, (uint32_t)RL_NS_CREATE);
    (void)PRINTF("Nameservice announce sent.\r\n");
```

图 49

（4） 等待接收握手协议数据，并循环收发乒乓信息。

```c
#ifdef RPMSG_LITE_MASTER_IS_LINUX
    /* Wait Hello handshake message from Remote Core. */
    (void)rpmsg_queue_recv(my_rpmsg, my_queue, (uint32_t *)&remote_addr, helloMsg, sizeof(helloMsg), ((void *)0),
                           RL_BLOCK);
#endif /* RPMSG_LITE_MASTER_IS_LINUX */

    while (msg.DATA <= 100U)
    {
        (void)PRINTF("Waiting for ping...\r\n");
        (void)rpmsg_queue_recv(my_rpmsg, my_queue, (uint32_t *)&remote_addr, (char *)&msg, sizeof(THE_MESSAGE),
                               ((void *)0), RL_BLOCK);
        msg.DATA++;
        (void)PRINTF("Sending pong...\r\n");
        (void)rpmsg_lite_send(my_rpmsg, my_ept, remote_addr, (char *)&msg, sizeof(THE_MESSAGE), RL_BLOCK);
    }
```

图 50

（5） 信息以乒乓形式结束后注销 RPMsg 相关资源，并进入死循环。

```
(void)PRINTF("Ping pong done, deinitializing...\r\n");

(void)rpmsg_lite_destroy_ept(my_rpmsg, my_ept);
my_ept = ((void *)0);
(void)rpmsg_queue_destroy(my_rpmsg, my_queue);
my_queue = ((void *)0);
(void)rpmsg_ns_unbind(my_rpmsg, ns_handle);
(void)rpmsg_lite_deinit(my_rpmsg);
my_rpmsg = ((void *)0);
msg.DATA = 0U;

(void)PRINTF("Looping forever...\r\n");

/* End of the example */
for (;;)
{
}
```

图 51

2.3 rpmsg_lite_str_echo_rtos 案例

2.3.1 案例说明

案例功能：本案例主要演示使用 RPMsg-lite 框架进行核间通信。实现在 Cortex-A53 核心发送"Tronlong"字符串到 Cortex-M7 核心，Cortex-M7 核心接收到"Tronlong"字符串后将此消息回发到 Cortex-A53 核心，Cortex-A53 核心接收并校验消息。

程序原理流程图如下所示：

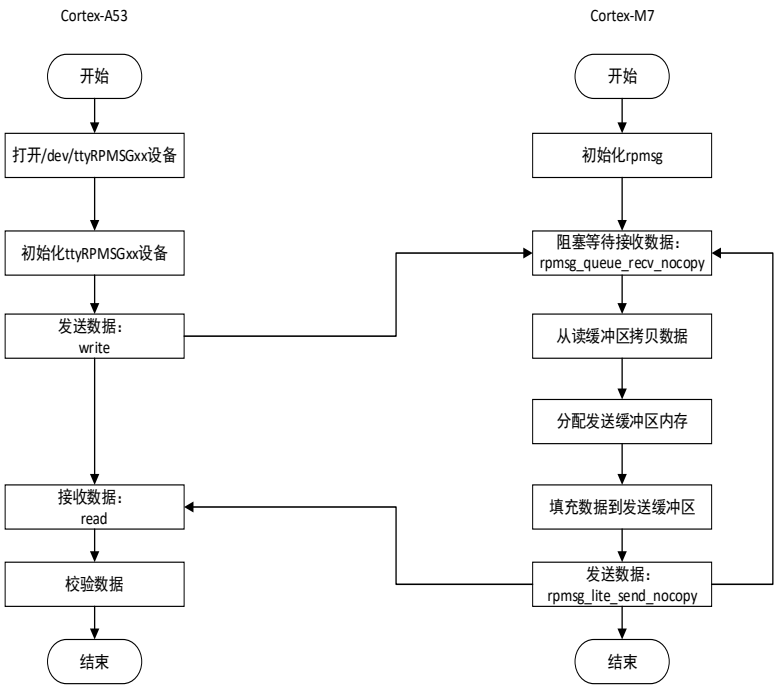

图 52

i.MX8MP 的 Cortex-A53 核心与 Cortex-M7 核心基于 RPMsg-lite 框架实现核间通信。本案例 Cortex-A53 通过加载 imx_rpmsg_tty 驱动向内核注册 TTY 终端设备，Linux 应用层通过访问"/dev/ttyRPMSGxx"设备节点从而间接调用 RPMsg-lite 接口实现与 Cortex-M7 核心通信。

系统框图如下图所示。

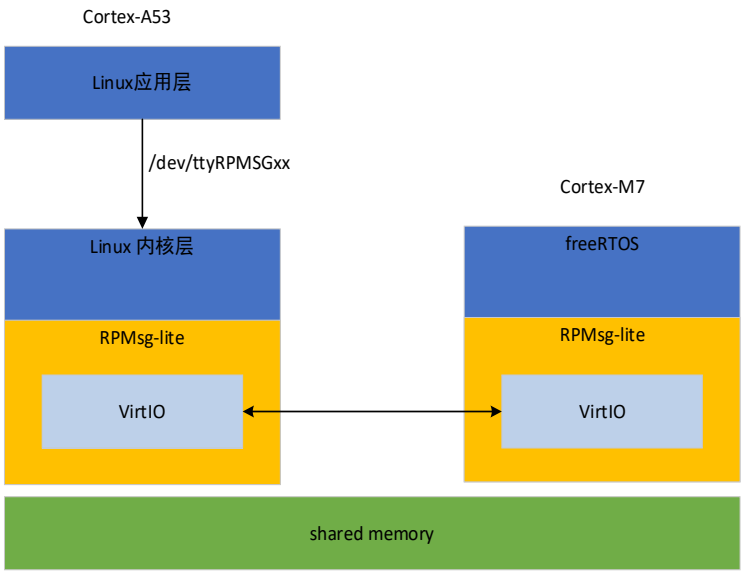

图 53

2.3.2 案例测试

（1） 加载运行 Cortex-M7 核心程序镜像文件

案例 bin 目录下分别提供了 TCM 版本程序镜像文件 rpmsg_lite_str_echo_rtos.bin 和 DDR 版本程序镜像文件 rpmsg_lite_str_echo_rtos_ddr.bin。请将 TCM 或 DDR 版本的 xxx.bin 程序镜像文件重命名为 m7_app.bin，并将其拷贝至 Linux 系统启动卡 BOOT 分区中。

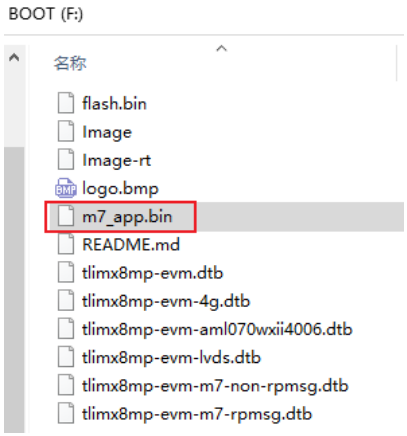

图 54

将 Linux 系统启动卡插至评估板 Micro SD 卡槽，启动方式选择拨码开关拨至 1100(1~4)，此档位为 Micro SD 卡启动模式。使用 USB 串口线将评估板的 USB TO UART2 串口连接至 PC 机的 USB 接口，使用 RS232 交叉串口母母线、USB 转 RS232 公头串口线将评估板的 RS232 UART4 串口连接至 PC 机的 USB 接口，如下图所示。

其中 USB TO UART2 作为 Cortex-A53 核心调试串口，RS232 UART4 作为 Cortex-M7 核心调试串口。

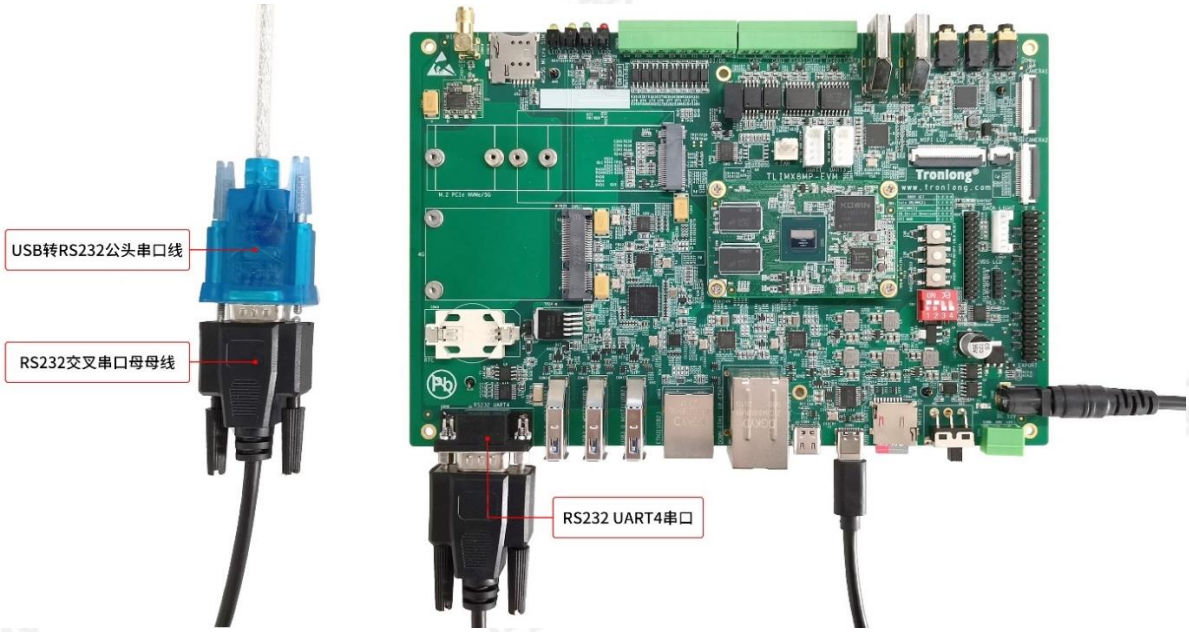

图 55

系统默认是 TCM 版本 Cortex-M7 核心程序镜像文件的运行环境，评估板上电启动，在 U-Boot 程序运行时，RS232 UART4 调试串口打印如下类似信息。

```
RPMSG String Echo FreeRTOS RTOS API Demo...
Nameservice sent, ready for incoming messages...
```

图 56

如需运行 DDR 版本程序镜像文件，请参考如下步骤进行设置，否则请忽略此内容。

评估板上电启动，在 U-Boot 倒计时结束之前按下空格键进入 U-Boot 命令行模式，在 U-Boot 命令行执行如下命令，设置 DDR 版本的 Cortex-M7 核心程序镜像文件运行环境。

U-Boot=> setenv m7_run_mem DDR

U-Boot=> saveenv

U-Boot=> reset

```
u-boot=>
u-boot=> setenv m7_run_mem DDR
u-boot=> saveenv
Saving Environment to MMC... Writing to MMC(2)... OK
u-boot=> reset
```

图 57

评估板重启，在 U-Boot 程序运行时，RS232 UART4 调试串口打印如下类似信息。

```
RPMSG String Echo FreeRTOS RTOS API Demo...
Nameservice sent, ready for incoming messages...
```

图 58

（2） 功能测试

进入评估板文件系统后，执行如下命令加载 imx_rpmsg_tty 模块，启动核间通信。

Target# modprobe imx_rpmsg_tty

```
root@IMX8MP-Tronlong:~# modprobe imx_rpmsg_tty
root@IMX8MP-Tronlong:~# [   81.988809] imx_rpmsg_tty virtio0.rpmsg-virtual-tty-channel-1.-1.30: new channel: 0x400 -> 0x1e!
[   81.988969] Install rpmsg tty driver!

root@IMX8MP-Tronlong:~#
```

图 59

Linux 系统加载模块后，会发送数据"hello world"至 Cortex-M7 核心。Cortex-M7 核心接收到数据后，在 RS232 UART4 调试串口将数据进行打印。

```
RPMSG String Echo FreeRTOS RTOS API Demo...

Nameservice sent, ready for incoming messages...
Get Message From Master Side : "hello world!" [len : 12]
```

图 60

驱动成功加载后，执行如下命令确认生成"/dev/ttyRPMSGxx"设备。

Target# ls /dev/ttyRPMSG*

```
root@IMX8MP-Tronlong:~# ls /dev/ttyRPMSG*
/dev/ttyRPMSG30
root@IMX8MP-Tronlong:~#
```

图 61

请将案例 bin 目录下的 rpmsg_lite_str_echo 文件拷贝至评估板文件系统，执行如下命令发送数据至 Cortex-M7 核心。

Target# ./rpmsg_lite_str_echo /dev/ttyRPMSG30

```
root@IMX8MP-Tronlong:~# ls rpmsg_lite_str_echo
rpmsg_lite_str_echo
root@IMX8MP-Tronlong:~# ./rpmsg_lite_str_echo /dev/ttyRPMSG30
send buf count: 8,send data: Tronlong
read buf count: 8,read data: Tronlong
Successful test .....
root@IMX8MP-Tronlong:~#
```

图 62

Cortex-M7 核心接收到数据后，在 RS232 UART4 调试串口将数据进行打印，同时将数据回发至 Cortex-A53。

```
RPMSG String Echo FreeRTOS RTOS API Demo...

Nameservice sent, ready for incoming messages...
Get Message From Master Side : "hello world!" [len : 12]
Get Message From Master Side : "Tronlong" [len : 8]
```

图 63

在 USB TO UART2 调试串口可查看 Cortex-M7 核心回发的数据。

```
root@IMX8MP-Tronlong:~# ls rpmsg_lite_str_echo
rpmsg_lite_str_echo
root@IMX8MP-Tronlong:~# ./rpmsg_lite_str_echo /dev/ttyRPMSG30
send buf count: 8,send data: Tronlong
read buf count: 8,read data: Tronlong
Successful test .....
root@IMX8MP-Tronlong:~#
```

图 64

2.3.3 案例编译

（1） RTOS 程序编译

请根据《Cortex-M7 核心开发手册》文档的“Cortex-M7 核心案例编译”章节将案例 project 目录下的程序源码进行编译，生成程序镜像文件。

（2） Linux 程序编译

请将产品资料“4-软件资料\Demo\openamp-demos\rpmsg_lite_str_echo_rtos\src\”下的文件拷贝至 Ubuntu 工作目录。

进入案例应用程序源码目录，执行如下命令进行编译，编译完成将会在当前目录下生成可执行文件。

Host# source /home/tronlong/SDK/environment-setup-armv8a-poky-linux

Host# make

```
tronlong@tronlong-virtual-machine:~/Demo/openamp-demos/rpmsg_lite_str_echo_rtos/src$ ls
Makefile  rpmsg_lite_str_echo.c
tronlong@tronlong-virtual-machine:~/Demo/openamp-demos/rpmsg_lite_str_echo_rtos/src$ source /home/t
ronlong/SDK/environment-setup-armv8a-poky-linux
tronlong@tronlong-virtual-machine:~/Demo/openamp-demos/rpmsg_lite_str_echo_rtos/src$ make
aarch64-poky-linux-gcc  -march=armv8-a+crc+crypto -fstack-protector-strong  -O2 -D_FORTIFY_SOURCE=2
 -Wformat -Wformat-security -Werror=format-security --sysroot=/home/tronlong/SDK/sysroots/armv8a-po
ky-linux -Wall -O2 rpmsg_lite_str_echo.c -o rpmsg_lite_str_echo
tronlong@tronlong-virtual-machine:~/Demo/openamp-demos/rpmsg_lite_str_echo_rtos/src$ ls
Makefile  rpmsg_lite_str_echo  rpmsg_lite_str_echo.c
tronlong@tronlong-virtual-machine:~/Demo/openamp-demos/rpmsg_lite_str_echo_rtos/src$
tronlong@tronlong-virtual-machine:~/Demo/openamp-demos/rpmsg_lite_str_echo_rtos/src$
```

图 65

2.3.4 关键代码

Linux 应用程序

（1） 打开"/dev/ttyRPMSG30"设备节点。

```
35    if((fd = open(argv[1], O_RDWR | O_NOCTTY )) < 0)
36    {
37        printf("open %s is error.\n",argv[1]);
38        return -1;
39    }
```

图 66

（2） 初始化"/dev/ttyRPMSG30"设备节点。

```
41    /* /dev/ttyRPMSG30 init */
42    tcgetattr(fd, &opt);
43    cfsetispeed(&opt, B115200);
44    cfsetospeed(&opt, B115200);
45    tcflush(fd, TCIOFLUSH);
46
47    opt.c_cflag |= (CLOCAL | CREAD);
48    opt.c_cflag &= ~CSIZE;
49    opt.c_cflag &= ~CRTSCTS;
50    opt.c_cflag |= CS8;
51    opt.c_iflag |= IGNPAR;
52    opt.c_cflag &= ~CSTOPB;
53    opt.c_oflag = 0;
54    opt.c_lflag = 0;
55
56    tcsetattr(fd, TCSANOW, &opt);
```

图 67

（3） 收发数据。

```
58    /* read and write tests */
59    count = write(fd, send_buf, strlen(send_buf));
60    printf("send buf count: %d,send data: %s\n", count, send_buf);
61    sleep(0.5);
62    count = read(fd, read_buf, sizeof(read_buf));
63    printf("read buf count: %d,read data: %s\n", count, read_buf);
```

图 68

（4） 校验数据。

```
    /* parity data */
    while (read_buf[i])
    {
        if(send_buf[i] != read_buf[i]){
            printf("Test error....\n");
            close(fd);
            return -1;
        }
        i++;
    }
```

图 69

Cortex-M7 核心关键代码

程序主体位于 main 函数中，定义位于 rpmsg_lite_str_echo_rtos.c 中。

（1） 在 main 函数中初始化相关资源，然后创建 app_task 任务，并启动任务列表。

```
    /* Initialize standard SDK demo application pins */
    /* M7 has its local cache and enabled by default,
     * need to set smart subsystems (0x28000000 ~ 0x3FFFFFFF)
     * non-cacheable before accessing this address region */
    BOARD_InitMemory();

    /* Board specific RDC settings */
    BOARD_RdcInit();

    BOARD_InitBootPins();
    BOARD_BootClockRUN();
    BOARD_InitDebugConsole();

    copyResourceTable();

#ifdef MCMGR_USED
    /* Initialize MCMGR before calling its API */
    (void)MCMGR_Init();
#endif /* MCMGR_USED */

    app_create_task();
    vTaskStartScheduler();
```

图 70

（2） 在 app_task 任务中初始化 RPMsg-lite，并进行数据收发。

```
    my_rpmsg = rpmsg_lite_remote_init((void *)RPMSG_LITE_SHMEM_BASE, RPMSG_LITE_LINK_ID, RL_NO_FLAGS);
#endif /* MCMGR_USED */

    rpmsg_lite_wait_for_link_up(my_rpmsg);

    my_queue = rpmsg_queue_create(my_rpmsg);
    my_ept   = rpmsg_lite_create_ept(my_rpmsg, LOCAL_EPT_ADDR, rpmsg_queue_rx_cb, my_queue);
    (void)rpmsg_ns_announce(my_rpmsg, my_ept, RPMSG_LITE_NS_ANNOUNCE_STRING, RL_NS_CREATE);

    PRINTF("\r\nNameservice sent, ready for incoming messages...\r\n");
```

图 71

（3） 在 for 循环中实现数据收发。

```
112     for (;;)
113     {
114         /* Get RPMsg rx buffer with message */
115         result =
116             rpmsg_queue_recv_nocopy(my_rpmsg, my_queue, (uint32_t *)&remote_addr, (char **)&rx_buf, &len, RL_BLOCK);
117         if (result != 0)
118         {
119             assert(false);
120         }
121
122         /* Copy string from RPMsg rx buffer */
123         assert(len < sizeof(app_buf));
124         memcpy(app_buf, rx_buf, len);
125         app_buf[len] = 0; /* End string by '\0' */
126
127         if ((len == 2) && (app_buf[0] == 0xd) && (app_buf[1] == 0xa))
128             PRINTF("Get New Line From Master Side\r\n");
129         else
130             PRINTF("Get Message From Master Side : \"%s\" [len : %d]\r\n", app_buf, len);
131
132         /* Get tx buffer from RPMsg */
133         tx_buf = rpmsg_lite_alloc_tx_buffer(my_rpmsg, &size, RL_BLOCK);
134         assert(tx_buf);
135         /* Copy string to RPMsg tx buffer */
136         memcpy(tx_buf, app_buf, len);
137         /* Echo back received message with nocopy send */
138         result = rpmsg_lite_send_nocopy(my_rpmsg, my_ept, remote_addr, tx_buf, len);
139         if (result != 0)
140         {
141             assert(false);
142         }
143         /* Release held RPMsg rx buffer */
144         result = rpmsg_queue_nocopy_free(my_rpmsg, rx_buf);
145         if (result != 0)
146         {
147             assert(false);
148         }
149     }
```

图 72

更多帮助

销售邮箱：sales@tronlong.com

技术邮箱：support@tronlong.com

创龙总机：020-8998-6280

技术热线：020-3893-9734

创龙官网：www.tronlong.com

技术论坛：www.51ele.net

官方商城：tronlong.tmall.com